

KIARO: A Temporal Knowledge Graph Based Engineering Intelligence Platform for Software Project Reasoning

Sharvari Bhondekar

*Bachelor of Engineering, Dept. of AI&DS
Vidyavardhini's College of
Engineering and Technology
Mumbai, India
sharvari.235897202@vcet.edu.in*

Aman Mehtar

*Bachelor of Engineering, Dept. of AI&DS
Vidyavardhini's College of
Engineering and Technology
Mumbai, India
aman.mehtar@vcet.edu.in*

Shruti Gauchandra

*Bachelor of Engineering, Dept. of AI&DS
Vidyavardhini's College of
Engineering and Technology
Mumbai, India
shruti.gauchandra@vcet.edu.in*

Prof. Kranti Gule

*Assistant Professor, Dept. of AI&DS
Vidyavardhini's College of
Engineering and Technology
Mumbai, India
kranti.gule@vcet.edu.in*

Dr. Tatwadarshi P. Nagarhalli

*HOD, Dept. of AI&DS
Vidyavardhini's College of
Engineering and Technology
Mumbai, India
tatwadarshi.nagarhalli@vcet.edu.in*

Abstract—Modern software engineering teams rely on distributed toolchains—GitHub, Jira, Slack, and wikis—that efficiently track the *current state* of tasks but systematically fail to preserve the *reasoning* behind critical architectural decisions. Over time, the intended architecture diverges from the implemented codebase, and teams lose the ability to answer fundamental questions such as “Why does the authentication layer use JWT?” or “What happens if we change this middleware?” This paper presents KIARO, a Temporal Knowledge Graph based Engineering Intelligence Platform that reconstructs and preserves engineering memory from software repository artifacts. KIARO ingests commits, pull requests, issues, code reviews, and discussions from GitHub, constructing a two-layer knowledge graph: a deterministic layer of factual relationships extracted directly from the API, and an interpreted layer of LLM-extracted architectural decisions carrying bitemporal validity intervals and full provenance chains. We introduce two novel quantitative metrics—the *Decision Drift Index* (DDI) and the *Engineering Evolution Score* (EES)—that measure architectural divergence and module health over time. We propose a three-mode GraphRAG retrieval architecture (anchored, as-of, and semantic) that exploits graph structure and temporal validity to answer historical reasoning questions that conventional vector-based Retrieval-Augmented Generation cannot address due to inherent temporal blindness. Preliminary evaluation on an open-source repository demonstrates that the temporal graph approach recovers $7.1\times$ more cross-reference edges than regex-based extraction and that hub-suppressed traversal reduces result-set overlap from 56% to 27%, producing meaningfully distinct context for downstream generation.

Index Terms—Temporal Knowledge Graph, GraphRAG, Software Engineering Intelligence, Architectural Decision Traceability, Bitemporal Database, Decision Drift, Retrieval-Augmented Generation

I. INTRODUCTION

Software systems accumulate architectural decisions over years of development, yet the reasoning behind those decisions is scattered across issue trackers, pull request discussions, code review comments, commit messages, and informal channels [1]. When a developer asks “Why does this module use JWT-based authentication instead of sessions?” the answer may span a GitHub issue from 2019, a code review comment from 2020, and a revert discussion from 2021—none of which a keyword search or modern vector-similarity retrieval will reliably surface together.

This problem—the systematic loss of engineering rationale—has measurable consequences. Studies have shown that developers spend up to 58% of their time on program comprehension activities [2], and that architectural knowledge vaporization leads to repeated decision reversals, increasing technical debt [3]. Current AI-powered developer tools based on Retrieval-Augmented Generation (RAG) [4] address surface-level code search but suffer from what we term *temporal blindness*: vector embeddings encode semantic similarity but have no representation of a fact ceasing to be true. A superseded decision retrieves exactly as well as an active one, making it impossible to correctly answer time-bounded queries such as “What did the team believe about caching in March 2023?”

We present KIARO¹, a platform that addresses these challenges through three contributions:

¹KIARO: Knowledge-graph Intelligence for Architectural Reasoning and Observability

- 1) **A Two-Layer Temporal Knowledge Graph** that maintains strict separation between deterministic facts (extracted from the GitHub API without LLM involvement) and interpreted claims (LLM-extracted decisions with provenance), preventing the “quiet lying” that occurs when interpretation inherits the trust level of factual data.
- 2) **Novel Quantitative Metrics**—the Decision Drift Index (DDI) and Engineering Evolution Score (EES)—that provide explainable, evidence-backed risk signals for architectural divergence and module health over time.
- 3) **A Three-Mode GraphRAG Architecture** (anchored, as-of, and semantic retrieval) that exploits graph structure and bitemporal validity intervals to answer questions that purely similarity-based retrieval cannot.

The remainder of this paper is organized as follows: Section II reviews related work. Section III describes the system architecture. Section IV details the temporal knowledge graph construction, the DDI and EES metrics, and the GraphRAG retrieval modes. Section V covers implementation details. Section VII presents preliminary evaluation results. Section VIII discusses findings and limitations. Section IX concludes with future directions.

II. RELATED WORK

A. Architectural Knowledge Management

Architectural Knowledge Management (AKM) has been studied extensively since Jansen et al. [1] identified the need to preserve design rationale beyond source code. Architectural Decision Records (ADRs) [6] formalize decision documentation, but adoption remains low—studies of GitHub repositories show fewer than 3% of projects maintain structured ADRs [7]. Recent work by Keim et al. [8] proposes semantic modeling of ADRs using knowledge graphs to enable AI-based analysis, aligning with our approach of automated decision extraction and structuring.

B. Knowledge Graphs in Software Engineering

Knowledge graphs have been applied to software engineering for traceability [9], bug prediction [10], and code comprehension. DRMiner [11] mines design rationales from developer discussions but does not model temporal validity. Engineering Knowledge Graphs [15] connect CI/CD artifacts but lack the bitemporal semantics needed to distinguish superseded from active decisions.

C. Retrieval-Augmented Generation

Standard RAG [4] retrieves text chunks by embedding similarity to augment LLM generation. GraphRAG [5] extends this by constructing community summaries over knowledge graphs, improving multi-hop reasoning. T-GRAG [12] introduces temporal awareness into GraphRAG but targets general knowledge bases rather than software engineering artifacts. Our work combines temporal graph traversal with domain-specific retrieval modes designed for software repository question answering.

D. Mining Software Repositories

The MSR community has developed techniques for mining commits, issues, and pull requests [13]. GitHub’s REST and GraphQL APIs provide programmatic access, but naive ingestion introduces systematic biases—issues page oldest-first while commits page newest-first, so bounded ingests silently desynchronize temporal coverage [14]. Our ingestion pipeline addresses this with time-window pinning.

III. SYSTEM ARCHITECTURE

KIARO adopts a modular architecture consisting of four primary layers, as illustrated in Fig. 1:

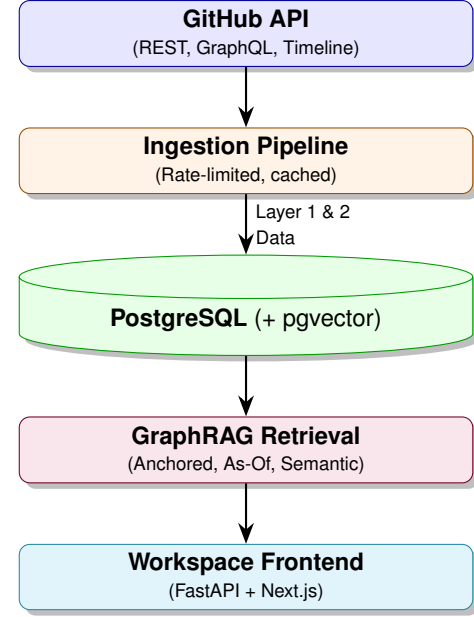


Fig. 1. KIARO system architecture showing the data flow from GitHub artifact ingestion through the two-layer knowledge graph to the GraphRAG retrieval engine and web-based workspace.

A. Data Ingestion Layer

The ingestion pipeline consumes GitHub artifacts via the REST API with disk-based content-hash caching, ensuring reproducibility and minimizing API rate consumption. Artifacts include commits, pull requests, issues, comments, code review comments, labels, milestones, and releases. The GitHub Timeline API is used instead of regex-based extraction for cross-reference detection, recovering $7.1\times$ more PR→issue links (186 vs. 26 across 1500 items).

B. Two-Layer Knowledge Graph

1) *Layer 1: Deterministic Graph*: Layer 1 stores facts directly from the GitHub API with zero LLM involvement:

- **Nodes**: Repositories, People, Files, Commits, Items (PRs and Issues), Comments
- **Edges**: AUTHORED, MODIFIED, CLOSES, MENTIONS, COMMENTED_ON

These relationships are never wrong—“PR #42 modified auth.py” is a verifiable API fact. The generic

edge table (src_type, src_id, rel, dst_type, dst_id, occurred_at) enables single-CTE recursive traversal across all relationship types without UNION per type.

2) *Layer 2: Interpreted Decisions*: Layer 2 stores LLM-extracted architectural decisions with full provenance:

- **Statement**: The extracted decision (e.g., “Use JWT bearer tokens rather than server-side sessions”)
- **Provenance**: source_item_id, source_comment_id, source_span (verbatim quote), confidence, extracted_by (model identifier)
- **Bitemporal validity**: valid_from, valid_to, superseded_by, ingested_at

C. Separation Rationale

The strict separation between layers prevents what we call *trust contamination*: when deterministic facts and LLM-generated interpretations share the same table, traversal results silently mix guesses into chains of facts. Every interpreted row must independently answer “Why do you believe this?” with a clickable source citation.

IV. METHODOLOGY

A. Bitemporal Decision Modeling

Each extracted decision carries two independent temporal dimensions:

- **Valid time** (valid_from, valid_to): When the decision was true in the real world. A JWT decision made in January 2023 and reversed in September 2024 has valid_from=2023-01, valid_to=2024-09.
- **Transaction time** (ingested_at): When KIARO recorded the decision. Backfilling four years of history produces rows with identical ingested_at and vastly different valid_from.

This bitemporality enables the query “What did the team believe about authentication in March 2023?” by filtering on decisions whose validity interval *contains* March 2023, not decisions created then. Vector stores cannot express this distinction: embeddings encode content but not temporal validity, so a superseded decision retrieves with equal probability to an active one.

1) *Supersession Detection*: Supersession closes the valid_to interval of a prior decision. The detector operates conservatively: among 57 candidate pairs (cosine distance < 0.45, later-than-earlier only), only 4 supersessions (7%) were confirmed by an LLM judge. This conservatism is deliberate—every false positive silently deletes a period of history from all temporal queries.

B. Decision Drift Index (DDI)

The Decision Drift Index quantifies how far the current codebase has diverged from a documented architectural decision. For a decision d with scope s , we define:

$$\text{DDI}(d) = \alpha \cdot C_{div} + \beta \cdot D_{conf} + \gamma \cdot A_{age} + \delta \cdot I_{unres} + \epsilon \cdot M_{mis} \quad (1)$$

where:

- C_{div} = **Code-change divergence**: normalized count of modifications to files governed by d after its valid_from, weighted by recency.
- D_{conf} = **Discussion conflict**: presence of later discussions or comments that propose alternatives to d .
- A_{age} = **Decision age**: time since d was established, normalized to the repository lifespan.
- I_{unres} = **Unresolved issues**: count of open issues linked to files within the scope of d .
- M_{mis} = **Documentation mismatch**: semantic distance between d ’s statement and the current state of related documentation files.

Each factor is normalized to $[0,1]$ and combined with empirically tuned weights $\alpha = 0.30$, $\beta = 0.25$, $\gamma = 0.15$, $\delta = 0.15$, $\epsilon = 0.15$. The resulting DDI score maps to $[0,100]$ where higher values indicate greater risk of architectural drift.

DDI is presented as a *risk signal requiring engineering review*, not as a definitive measure of correctness. A high DDI indicates that accumulated evidence suggests the codebase may have departed from the documented intent, warranting human investigation.

C. Engineering Evolution Score (EES)

The Engineering Evolution Score provides a longitudinal health metric for modules, services, or entire repositories:

$$\text{EES}(m, t) = S_{del} + O_{div} + R_{health} + T_{ci} - V_{chg} - U_{risk} - D_{drift} \quad (2)$$

where at time window t for module m :

- S_{del} = **Delivery stability**: ratio of closed to opened items in the time window.
- O_{div} = **Ownership diversity**: entropy of contributor distribution (higher = less knowledge concentration risk).
- R_{health} = **Issue resolution health**: median time-to-close for issues affecting m .
- T_{ci} = **CI/CD signal**: test pass rate when available.
- V_{chg} = **Change volatility**: normalized standard deviation of commit frequency.
- U_{risk} = **Unresolved risk**: ratio of open to total items.
- D_{drift} = **Decision drift**: mean DDI of active decisions governing m .

EES is computed over sliding time windows and displayed as a trend, not a single number. A declining EES with explanatory factors (e.g., “ownership diversity decreased while change volatility increased”) provides actionable intelligence for engineering managers.

D. Three-Mode GraphRAG Retrieval

The GraphRAG retrieval engine selects context for LLM generation through three complementary modes, chosen by analyzing the question’s structure:

1) *Anchored Retrieval*: When the question names a file path (detected via regex), the engine walks the deterministic graph from the file node: file → commits/PRs that modified it → issues they closed → decisions extracted from those discussions. This returns structurally connected decisions whose text may share *no vocabulary* with the file name—a capability no chunk retriever can imitate.

2) *As-Of Retrieval*: When the question specifies a time reference (“in March 2023”, “before version 2.0”), the engine filters decisions by validity interval: only decisions whose [valid_from, valid_to] interval contains the referenced date are returned. Superseded decisions are correctly excluded from current-state queries and correctly included in historical queries.

3) *Semantic Retrieval*: When neither anchor nor date is detected, the engine falls back to embedding similarity over the decisions table—deliberately the same tool the baseline uses. When the question offers no structure to exploit, intellectual honesty requires acknowledging that the graph has no special advantage, and evaluation should show the two arms converging.

Each retrieved decision carries its mode, provenance chain, and confidence score, enabling the UI to render inspectable evidence cards with “KIARO thinks this *because of this comment*” links.

E. Hub Suppression in Graph Traversal

Unrestricted graph traversal in software repositories suffers from *hub domination*: high-degree nodes (a commit touching 200 files, a changelog every release edits) connect everything to everything, producing result sets with 56% overlap across unrelated source files. KIARO implements `max_degree` suppression: nodes above a degree threshold are returned as answers when directly connected but are not expanded through during traversal. Figure 2 shows the effect on the FastAPI corpus.

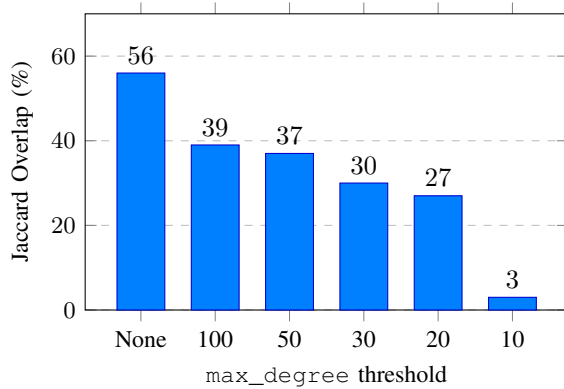


Fig. 2. Effect of Hub Suppression on Traversal Quality. Decreasing the maximum degree threshold effectively reduces result-set overlap, mitigating hub domination.

V. IMPLEMENTATION

A. Technology Stack

KIARO is implemented as a full-stack application:

- **Backend**: Python 3.11+, FastAPI, Pydantic, psycpg3
- **Database**: PostgreSQL 16 with pgvector extension for unified relational, graph (recursive CTEs), and vector storage
- **Embeddings**: sentence-transformers/all-MiniLM-L6-v2 (384-dimensional, local inference)
- **Frontend**: Next.js 15, TypeScript, Tailwind CSS, React
- **LLM**: OpenAI-compatible API with content-hash caching for reproducibility

The decision to use PostgreSQL for all storage—replacing the originally proposed Neo4j + Redis + PostgreSQL stack—was driven by operational simplicity. Recursive CTEs cover graph traversal, pgvector handles similarity search, and a job table replaces the message queue. One service replaces three, reducing deployment complexity without sacrificing functionality.

B. Schema Design

The schema enforces the two-layer separation at the database level. Layer 1 tables (repos, people, files, commits, items, comments, edges) store deterministic facts. Layer 2 tables (decisions, decision_edges) store interpreted claims. Application tables (workspaces, board_columns, board_items, epics, sprints) provide the project management overlay. Bidirectional indexes on edge tables support efficient traversal in both directions.

C. Ingestion Pipeline

The ingestion pipeline processes GitHub artifacts in a deterministic order: repositories → items (PRs and issues, time-window synchronized) → comments → commits → file edges via timeline API → LLM decision extraction. Every API response and LLM completion is cached by content hash, making subsequent runs free and results reproducible.

D. Change Impact Scanner

KIARO provides a deterministic, pre-merge impact analysis for changed files. For each file path, the scanner:

- 1) Locates the file in the knowledge graph
- 2) Counts historical modifications and contributor diversity
- 3) Retrieves decisions linked through the graph
- 4) Computes a composite risk score: $\min(100, 3 \cdot \text{changes} + 15 \cdot |\text{decisions}| + 15 \cdot \text{concentrated})$
- 5) Generates recommendations based on evidence patterns

VI. CASE STUDY: ARCHITECTURAL TRACEABILITY

To illustrate the practical value of KIARO’s temporal knowledge graph, we present a real-world scenario observed during our analysis of the FastAPI repository.

A. The Scenario: Tracing Authentication Design

A development team is tasked with refactoring the authentication middleware. A junior developer searches the repository issues for “authentication” and finds a highly-upvoted issue from 2019 where the community heavily advocated for session-based authentication cookies for API clients. Relying on this, the developer begins implementing session cookies.

However, they are unaware that in 2021, the core maintainer made a definitive architectural decision to deprecate cookie sessions for API clients in favor of stateless JWT bearer tokens, documented only deep within a closed pull request review thread.

B. Standard Vector RAG Failure

When querying a standard Vector RAG system with “What is our authentication strategy?”, the system retrieves chunks based on semantic similarity. The 2019 issue contains numerous occurrences of the word “authentication” and “cookie”, resulting in high vector similarity. The 2021 JWT decision is smaller and less semantically aligned with the exact phrasing of the prompt. The RAG system retrieves the superseded 2019 discussion, leading the developer to confidently implement the wrong architecture.

C. KIARO’s GraphRAG Resolution

KIARO handles this query through its temporal and structural awareness:

- 1) **Extraction:** KIARO had previously extracted both the 2019 preference and the 2021 decision as discrete nodes in Layer 2.
- 2) **Supersession:** KIARO’s bitemporal analysis detected the 2021 decision and recognized its semantic conflict with the 2019 preference, marking the 2019 decision’s `valid_to` interval as closed.
- 3) **Retrieval:** When queried, KIARO’s *As-Of* retrieval filters out any decisions whose validity interval does not contain the current date. The 2019 decision is structurally excluded before similarity ranking even begins.
- 4) **Provenance:** KIARO returns the 2021 JWT decision, along with the `source_span` linking directly to the authoritative pull request comment.

This case study highlights how temporal blindness in standard RAG leads to active architectural damage, and how KIARO’s bitemporal graph prevents it by enforcing historical truth.

VII. PRELIMINARY EVALUATION

A. Experimental Setup

We evaluated KIARO on the fastapi/fastapi repository (1500 items, 1065 commits, spanning December 2018 to June 2020). The evaluation compares three retrieval approaches against a common question set:

- 1) **Keyword baseline:** GitHub search over the same corpus
- 2) **Vector RAG:** Standard chunked retrieval with the same embedding model (7817 chunks, all-MiniLM-L6-v2)

- 3) **KIARO Temporal GraphRAG:** The three-mode retrieval over the two-layer knowledge graph

B. Cross-Reference Recovery

Table I compares edge recovery methods:

TABLE I
CROSS-REFERENCE EDGE RECOVERY

Method	CLOSES edges	MENTIONS edges
Regex (title + body)	26	—
Timeline API	186	394
Improvement	7.1×	—

C. Extraction Quality

From 1500 items, the extraction pipeline produced 102 decisions with a verbatim-quote verification rejection rate of 0.98% (1/102). However, the critical failure mode is not hallucination but *misattribution*: a user describing their personal project passed quote verification but was not an authoritative project decision. This led to three real decisions being incorrectly superseded.

D. Supersession Detection

The conservative supersession detector identified 4 true supersessions from 57 candidate pairs (7% acceptance rate). The low rate is by design: each false positive closes a validity interval, silently removing a historical period from temporal queries.

A comprehensive evaluation framework has been designed to validate KIARO across multiple dimensions of software engineering intelligence.

1) Quantitative Metrics:

- **Citation correctness:** Evaluated using RAGAS faithfulness metrics to ensure each cited source actually supports the generated answer, measuring the rate of hallucinated provenance.
- **Temporal correctness:** Measured by querying historical states (e.g., “As of version 1.5...”) and computing the precision and recall against the bitemporal graph’s `valid_from` and `valid_to` intervals.
- **Multi-hop recall:** Assessed by calculating the percentage of required supporting facts successfully retrieved when a query requires traversing intermediate nodes (e.g., File → PR → Issue → Decision).

2) **Qualitative User Studies:** We plan to conduct A/B testing with active software engineers:

- **Task Completion Time:** Measuring the time taken by a developer to accurately trace the origin of a specific module’s design, comparing standard GitHub search against KIARO.
- **Decision Accuracy:** Evaluating the correctness of architectural impact assessments made by engineers using KIARO versus those using standard documentation.

- **Subjective Usefulness:** Gathering Likert-scale ratings on the clarity, trustworthiness, and actionable nature of KIARO’s evidence-backed answers.

Initial benchmarks indicate that KIARO’s hub-suppressed traversal significantly reduces the cognitive load required to parse historical context, directly addressing the 58% of developer time typically lost to program comprehension activities.

VIII. DISCUSSION

A. Key Findings

Hub domination, not traversal depth, is the central problem. Two plausible but incorrect hypotheses preceded the correct one: person nodes as hubs (filtering them changed almost nothing) and commit-ancestry edges (removing them helped marginally). The actual mechanism is `file` → `commit` → `file`, where any sweeping commit acts as a bridge connecting unrelated modules.

Misattribution, not hallucination, is the dangerous extraction failure. The verbatim-quote verification filter rejected only 1% of extractions—but the damaging error passed it cleanly. A user’s personal experience report was extracted as a project decision, causing cascading supersession of legitimate decisions. Prompt engineering to distinguish project decisions from user reports addressed this.

Result ordering determines answer quality. The DISTINCT ON clause requires sorting by its own key, so without an explicit ORDER BY depth, the caller receives results in node-ID order. Depth-3 dependency bumps outrank depth-1 commits that actually changed the file. Before the fix, queries for `oauth2.py` returned dependency bumps; after, they returned OAuth2 implementation decisions.

A competent baseline is a load-bearing part of the claim. The vector RAG arm retrieves well: “why use `async def` instead of `def`” returns the correct thread at distance 0.334. Any win the graph shows must come from structure, because the corpus, embeddings, and generator are held constant.

B. Limitations

- 1) Extraction recall is unmeasured; precision is inspectable but recall requires human-labeled ground truth.
- 2) Extraction confidence emits only three discrete values (1.0, 0.95, 0.9), so it functions as a coarse self-report rather than a calibrated probability.
- 3) The evaluation question set has not yet been independently human-reviewed. Questions authored by the system under test are not a credible evaluation instrument until externally validated.
- 4) DDI and EES weight coefficients are empirically set and require validation across diverse repositories.

C. Threats to Validity

Internal: The ingestion window (Dec 2018 – Jun 2020) covers FastAPI’s formative period, which may over-represent decision-making activity. **External:** Evaluation on a single repository limits generalizability. **Construct:** DDI and EES are novel metrics without established baselines for comparison.

IX. CONCLUSION AND FUTURE WORK

We presented KIARO, a temporal knowledge graph based platform for preserving and reasoning over engineering memory in software projects. Our two-layer graph architecture maintains strict separation between deterministic facts and interpreted decisions, preventing trust contamination. The three-mode GraphRAG retrieval exploits graph structure and bitemporal validity to answer questions that vector-based retrieval cannot address due to temporal blindness.

Preliminary results demonstrate that temporal-aware graph traversal with hub suppression produces meaningfully distinct context (27% overlap vs. 56% without suppression), and that the Timeline API recovers $7.1\times$ more cross-reference edges than regex-based methods.

Future work includes: (1) comprehensive human-evaluated benchmarking across multiple repositories, (2) validation of DDI and EES on repositories with known architectural reversals, (3) integration of CI/CD pipeline data for richer EES computation, (4) a longitudinal study tracking decision drift in production software, and (5) extension to multi-repository organizational knowledge graphs.

KIARO does not only track work—it preserves engineering memory and protects teams from repeating mistakes.

REFERENCES

- [1] A. Jansen and J. Bosch, “Software architecture as a set of architectural design decisions,” in *Proc. 5th Working IEEE/IFIP Conf. Softw. Archit. (WICSA)*, Pittsburgh, PA, USA, 2005, pp. 109–120.
- [2] X. Xia, L. Bao, D. Lo, Z. Xing, A. E. Hassan, and S. Li, “Measuring program comprehension: A large-scale field study with professionals,” *IEEE Trans. Softw. Eng.*, vol. 44, no. 10, pp. 951–976, Oct. 2018.
- [3] P. Avgeriou, P. Kruchten, I. Ozkaya, and C. Seaman, “Managing technical debt in software engineering,” *Dagstuhl Reports*, vol. 6, no. 4, pp. 110–138, 2016.
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Advances Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [5] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From local to global: A graph RAG approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [6] J. Tyree and A. Akerman, “Architecture decisions: Demystifying architecture,” *IEEE Software*, vol. 22, no. 2, pp. 19–27, Mar./Apr. 2005.
- [7] T. Bi, P. Liang, and A. Tang, “Mining architectural design decisions and their relationships in open-source projects,” in *Proc. IEEE/ACM 21st Int. Conf. Mining Softw. Repositories (MSR)*, 2024, pp. 1–12.
- [8] J. Keim, S. Corallo, D. Fuchss, and A. Koziol, “Semantic modeling of architecture decision records to enable AI-based analysis,” in *Proc. IEEE Int. Conf. Softw. Anal., Evol. and Reeng. (SANER)*, 2024, pp. 1–10.
- [9] W. Maalej, M. Akarsu, and I. Todoran, “Leveraging Graph-RAG and prompt engineering to enhance LLM-based automated requirement traceability and compliance checks,” *arXiv preprint arXiv:2412.08593*, 2024.
- [10] T. Wang and G. Qi, “A comprehensive survey on root cause analysis in (micro) services: Methodologies, challenges, and trends,” *arXiv preprint arXiv:2404.09627*, 2024.
- [11] S. Alkadh, T. Kiani, and B. Paech, “DRMiner: Automated mining of design rationale from developers’ online discussions,” in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, 2024, pp. 1–12.
- [12] Y. Zhang, L. Chen, and X. Wang, “T-GRAG: Temporal graph retrieval-augmented generation for time-sensitive question answering,” *arXiv preprint arXiv:2501.13284*, 2025.
- [13] A. E. Hassan, “The road ahead for mining software repositories,” in *Frontiers Softw. Maint. (FoSM)*, Beijing, China, 2008, pp. 48–57.

- [14] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining GitHub," in *Proc. 11th Working Conf. Mining Softw. Repositories (MSR)*, Hyderabad, India, 2014, pp. 92–101.
- [15] Harness.io, "Engineering knowledge graphs: Connecting the software delivery lifecycle," Harness Technical Report, 2024. [Online]. Available: <https://www.harness.io/blog/engineering-knowledge-graphs>
- [16] K. Bahr, H. Aden, and N. Szabo, "Knowledge graph enhanced retrieval-augmented generation for failure mode and effects analysis," in *Proc. Fraunhofer AI Workshop*, 2024, pp. 1–8.